

Case Study: Sudoku Solver Application

Objective

As part of the next interview stage, you are asked to develop a small application based on your Sudoku-solving algorithm.

Functional Requirements

Build an application that can be executed locally on a PC using Docker.

The application should support the following workflow:

1. **Import**
 - Users should be able to upload unsolved Sudoku puzzles.
 - The input format is your decision; for example, an Excel-based template may be used.
2. **Processing**
 - The uploaded puzzles should be processed using the existing Sudoku-solving algorithm.
 - Integration of the algorithm into the application is part of the task.
3. **Database**
 - Solved Sudoku puzzles should be stored in a database of your choice.
 - Users should be able to select previously solved Sudoku puzzles through the application interface.
4. **Export**
 - Users should be able to select a solved Sudoku puzzle and export it as an image file (PNG or JPG) with a single action.

Technical Requirements

- Use a modern web framework such as Django or Ruby on Rails.
- The application must be runnable locally using Docker.
- The database technology is your choice.
- You may use additional libraries or frameworks.

Documentation

Please provide a concise README containing the following information. Formal documentation is not required.

- Setup and execution instructions
- Architecture overview
- Key design decisions
- Assumptions and limitations

Presentation

Please prepare a short informal presentation of your solution. The presentation should cover at least:

- Overall system architecture
- Database design
- Key technical decisions
- Challenges encountered and how they were addressed

Creativity & Ownership

There is intentionally no single "correct" solution for this exercise. You are encouraged to challenge assumptions, propose alternative approaches, and introduce your own ideas where you believe they improve the solution. We are interested in understanding your reasoning and engineering trade-offs as much as the final implementation itself.